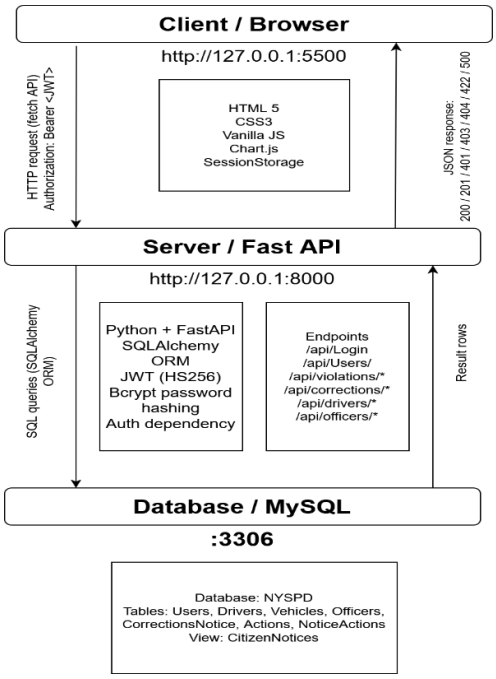
CMP: 2812 Full stack development
Name: Harrison Macdonald
Student Number: 28958802

CMP 2812 Full stack development
Assessment 3: Frontend development
Harrison Macdonald
28958802

1. Design

1.1 Architecture



(Figure 1.)

The frontend is built on HTML5, CSS3 and JavaScript. The code is organized into 4 folders; Admin, CSS, Driver, and js.

css/main.css – Defines global styles, CSS custom properties for theming and responsive design breakpoints for tablet and mobile devices.

css/driver.css – Contains styling specific to the driver portal including form layouts, dashboard tables, status badges and modal overlays.

css/admin.css – contains styling specific to the admin portal including stat cards, action cards, notice management tables, confirmation messages and chart visuals.

Js/api.js – Centralizes all API communications with FastAPI, using a core apiFetch() wrapper function handling authentication, error response and endpoint organization.

Js/auth.js – Manages user authentication, session storage in sessionStorage, page protection through requireDriverAuth() and requireAdminAuth(), and automatic token expiry checks every 60 seconds.

Js/validations.js – Provides client-side form validation with visual feedback through CSS classes and error messages.

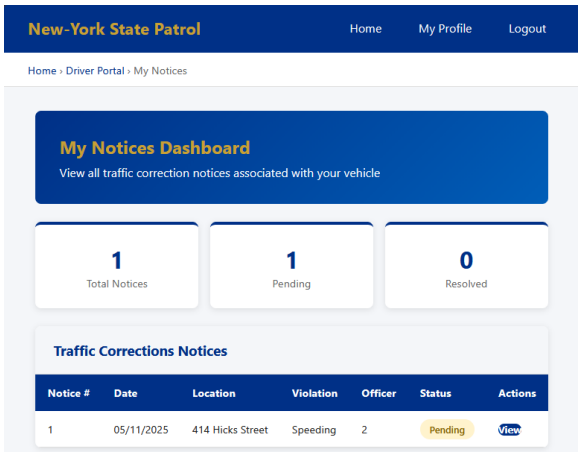
Js/ui.js – Contains shared UI utility functions for formatting and data transformation.

CMP: 2812 Full stack development
Name: Harrison Macdonald
Student Number: 28958802

1.2 User Interface (UI)

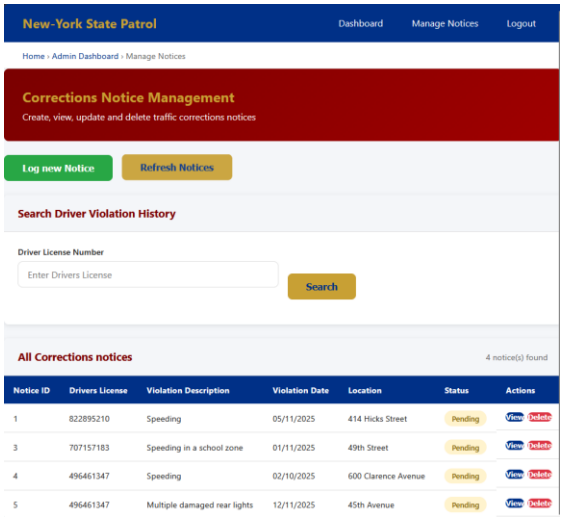
The frontend provides two user interfaces: a Driver portal for vehicle owners viewing traffic notices, and an admin portal for officers manage notices.

The driver portal consists of 3 pages. Login/register, Dashboard and the profile. Login/register allows existing users to authenticate and new users to create accounts. The dashboard displays statistics (Total notices, pending, resolved) and a table of violations. Users can click “view” to open a modal showing full notice details, the profile page allows users to update their name and address.



(figure 2)

The admin portal includes 2 pages. Login and Dashboard. The dashboard provides overall notice management: Stat cards (total notices, total officers, pending, resolved), quick action cards, a notices table with columns for Notice ID, Driver license, violation, date, location, and status, and search functionality to retrieve driver violation history by license number. Officers can create notices via modal form, view details, or delete with confirmation. Charts visualize notice distribution by status and violation type.



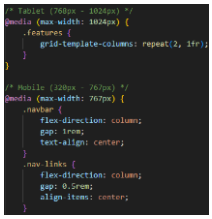
(figure 3)

1.3 Responsivity

A desktop-first approach was undertaken. This is because the target audience would be officers on computers inputting a notice to the system.

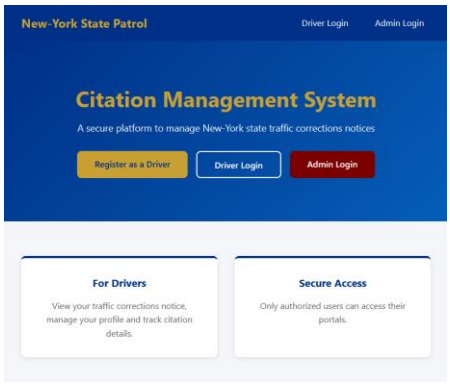
Two CSS media query breakpoints create a responsive design: tablet (768px-1024px) and mobile (320px-767px), with desktop (1025px and above) as the default layout.

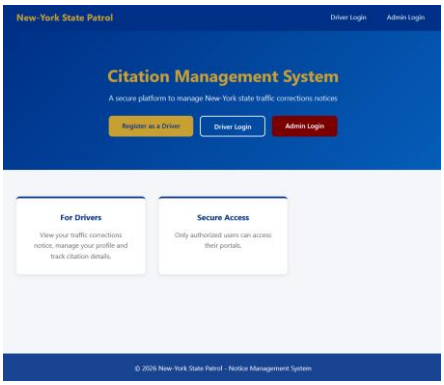
CMP: 2812 Full stack development
Name: Harrison Macdonald
Student Number: 28958802

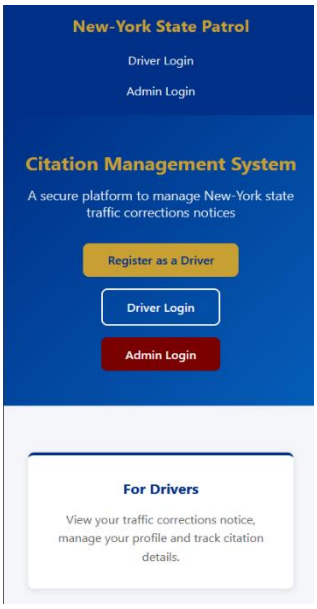


```
/* Tablet (768px - 1024px) */
@media (max-width: 1024px) {
    .features {
        grid-template-columns: repeat(2, 1fr);
    }
}

/* Mobile (320px - 767px) */
@media (max-width: 767px) {
    .navbar {
        flex-direction: column;
        gap: 1rem;
        text-align: center;
    }
    .nav-links {
        flex-direction: column;
        gap: 0.5rem;
        align-items: center;
    }
```







CSS Grid and Flexbox are used to create a responsive layout. Grid, used for the responsive table layouts in admin and driver portals, automatically wrapping elements without changing the HTML structure. Grid columns adapt to viewport size: 4 columns on desktop, 2 columns on tablet and 1 column on mobile, ensuring tables remain readable across all device sizes. Flexbox is used for 1D layouts like the navigation bar and button rows.

2. Implementation

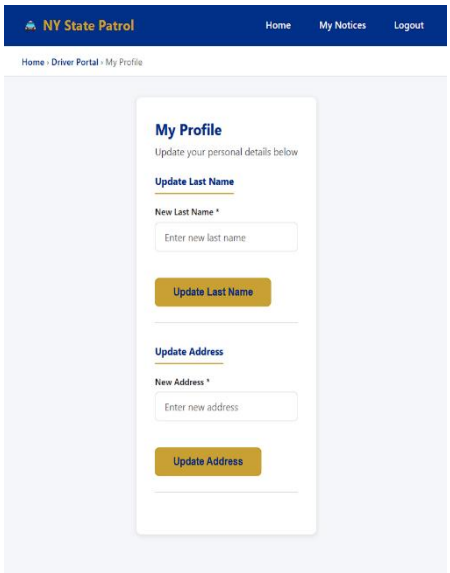
2.1 HTML Structure

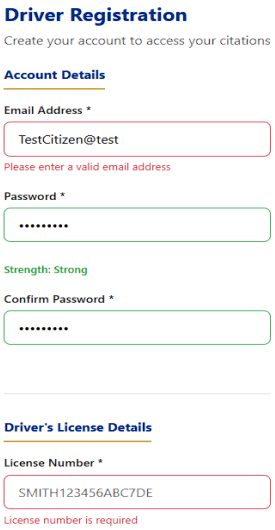
The HTML is structured using semantic elements to provide clear meaning and improve accessibility. Each page follows a consistent layout with a navigation bar, main content area and a footer.

Semantic HTML elements are used throughout: <nav> contains the navigation bar with links; <main> wraps the primary page content; <footer> contains hypothetical copyright information; <section> groups related content; <form> wraps all form inputs; and <table> with <thead> and <tbody> structure the notice tables on the dashboards.

```
<!-- Navigation -->
<nav class="navbar">
    <div class="nav-brand">
        <h1>New-York State Patrol</h1>
    </div>
    <div class="nav-links">
        <a href="../index.html">Home</a>
        <a href="profile.html">My Profile</a>
        <a href="#" onclick="logoutDriver()">Logout</a>
    </div>
</nav>

<!-- Breadcrumb -->
<div class="breadcrumb">
    <a href="../index.html">Home</a> &rsaquo;
    <a href="dashboard.html">Driver Portal</a> &rsaquo;
    <span>My Notices</span>
</div>
```





All form inputs have associated <label> elements with ‘for’ attributes linking to unique input ‘id’ values, ensuring accessibility for screen readers. Error message elements are pre-rendered below each input field to prevent layout shift when validation messages appear. Modal overlays use containers with flexbox centring to display dialogs for viewing notices, creating notices, and confirming deletions, keeping users on the same page without page reloads.

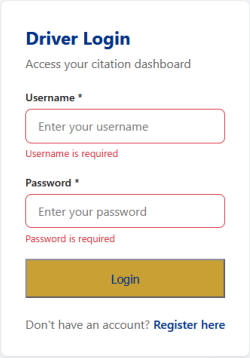
The application uses separate HTML files for each page: /driver/login.html, /driver/dashboard.html, /driver/profile.html, /admin/login.html, /admin/dashboard.html and /admin/notices.html. The navigation bar is a consistent component included on every page. Breadcrumb navigation appears below the nav bar on each page to show the hierarchy (e.g. “Home > Admin Dashboard > Manage Notices”). This structure keeps the codebase organized while avoiding code duplication for common elements.

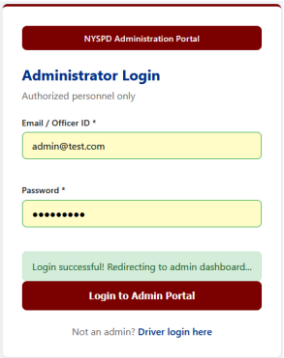
2.2 CSS and Responsive Techniques

The frontend consists of three files, main.css, admin.css and driver.css; main.css holds core theme choices and global styles. E.g. navbar, button, header and footer style is defined within main.css.

Custom variables are defined in main.css under root, background colours, button colours, text colours, shadow properties, radii, window maximum width. this is so that instead of typing the colour hex-codes I can reference the variable names, making maintaining the correct colours across the frontend simpler.

```
:root {
    --primary: #003087;          /* NYSPD navy blue */
    --secondary: #ffffff;
    --accent: #c8a034;           /* GOLD */
    --admin: #7b0000;            /* Dark Red */
    --background: #f4f6f9;
    --text: #222222;
    --text-light: #666666;
    --border: #dddddd;
    --shadow: 0 2px 8px rgba(0,0,0,0.1);
    --radius: 8px;
    --max-width: 1200px
}
```





Visual feedback is implemented through CSS classes. Form inputs use .error class (red border and text) for invalid fields and .success class (green border) for valid fields. Status badges use colour-coded backgrounds: yellow for pending, green for resolved and red for disputed. Loading indicators display a spinner animation during API calls. Modal overlays use semi-transparent backgrounds with firebox centring to display dialogs.

2.3 JavaScript Implementation

2.3.1 DOM Manipulation and Event handling

The frontend uses JavaScript dynamically manipulate the DOM and

```
function renderOfficers(officers) {
    const tbody = document.getElementById('officersBody');
    tbody.innerHTML = '';

    if (!officers || officers.length === 0) {
        tbody.innerHTML = '<tr><td colspan="6" style="text-align:center">No officers found</td></tr>';
        return;
    }

    officers.forEach(officer => {
        const row = document.createElement('tr');
        row.innerHTML = `
            <td>${officer.OfficerID || 'N/A'}</td>
            <td>${officer.FirstName || 'N/A'}</td>
            <td>${officer.LastName || 'N/A'}</td>
            <td>${officer.PersonnelNumber || 'N/A'}</td>
            <td>
                <button class="btn-small btn-warning" onclick="openOfficerModal()">
                    Edit Name
                </button>
            </td>
        `;
        tbody.appendChild(row);
    });
}
```

respond to user interactions through event listeners, enabling updates without page reloads.

DOM manipulation uses getElementById() to select form inputs and elements for updating. createElement() dynamically creates table rows when rendering notices from API responses, with InnerHTML populating them with data. classList methods toggle CSS classes for visual feedback: .error for invalid form fields, .success for valid fields, and display toggling for modal overlays and loading indicators.

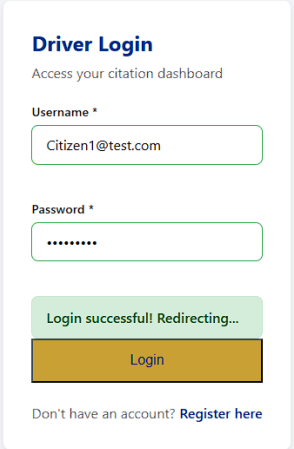
Event listeners respond to user interactions: blur events trigger real-time form validation, submit events prevent default form submission and handle it with JavaScript, and click events trigger actions like viewing or deleting notices. Event listeners are attached using inline onclick handlers in HTML and addEventListener() in JavaScript files.

2.3.2 API Integration and Data transformation

js/api.js acts as a central location for all communication with the FastAPI backend and contains the apiFetch() wrapper function. apiFetch() handles authentication by retrieving the JWT token from sessionStorage and adding it to the authorization header as a Bearer token for protected endpoints. If no token exists, the user is redirected to the login page. The function returns a standardized response object containing the HTTP status, success indicator, and data from the API.

The file also contains endpoint functions organized by feature type: authentication (loginUser(), registerUser()), violations (getMyViolations(), getViolationsByLicense()), drivers (updateDriverLastname(), updateDriverAddress(), deleteDriver()), officers (getAllOfficers(), updateOfficerLastname()), and vehicles (deleteVehicle)). Each endpoint function uses apiFetch() and returns consistent response objects, enabling uniform error handling and data transformation throughout the application.

2.3.3 Form Validation



Js/validation.js provides client-side form validation with real-time visual feedback. Prerequisite utility functions showError() and showSuccess() provide visual feedback by .error (red) or .success (green) CSS classes to inputs. validateRequired() checks if form fields are empty.

Password validation includes validatePassword(), which checks password strength by verifying minimum length, 1 uppercase letter, 1 number. validateConfirmPassword() ensures two password fields match. validateEmail() uses regex pattern matching to validate email format (/^[^\s@]+@[^\s@]+\.[^\s@]+$/).

validateRegistrationForm() calls all individual validators; validateEmail(), validatePassword(), ValidateConfirmPassword() and ValidateRequired() before allowing form submission. Only if all validation functions return true does the form submission proceed.

```
function validateRegistrationForm() {
    const results = [
        validateEmail('email'),
        validatePassword('password'),
        validateConfirmPassword('password', 'confirmPassword'),
        validateRequired('licenseNumber', 'License number'),
    ];
    return results.every(result => result === true);
}
```

2.3.4 State Management

js/auth.js manages user authentication state and session persistence. The JWT token and user role are stored in sessionStorage, which clears automatically when the browser closes, preventing tokens from persisting between sessions. Functions saveSession() and clearSession() manage storage after login and logout respectively. isLoggedIn() checks if a valid token exists, getRole() retrieves the current user's role, and decodeToken() extracts embedded user information from the JWT.

Page protection is enforced through requireDriverAuth() and requireAdminAuth() functions, which verify user roles on page load and redirect unauthorized users to the appropriate login page. Token expiry is checked every 60 seconds using checkTokenExpiry(), ensuring users are automatically logged out when sessions expire rather than only when making API calls. isTokenExpired() determines if the 30-minute token expiry time as elapsed by comparing the tokens exp claim to the current time.

2.3.5 Error handling

Error handling is implemented at multiple layers. The apiFetch() wrapper function in js/api.js handles HTTP error responses: 401 (Unauthorized) clears the session and redirects to the appropriate login page, 403 (Forbidden) throws a permission error, 404 (Not Found) throws a resource not found error, and 500 (Server Error) throws a server error message. All API calls wrap responses in try-catch blocks, catching network errors and displaying user-friendly error messages.

All async functions that call API's return standardized response objects containing a success flag, message, and data. This consistent response format enables uniform error handling across the application. API calls also display loading indicators during requests and hide them on completion, whether successful or failed. When API calls fail, error messages are displayed in message container, allowing users to understand what went wrong and retry operations.

2.4 JWT Authentication and CORS

When users log in, the /api/Login endpoint returns a JWT which the frontend decodes to extract the user's role and token expiry time. The token is stored in sessionStorage and included in the authorization header as a Bearer token for all projected API requests. This enables stateless authentication where the frontend carries the user's credentials in each request.

CORS (Cross-Origin Resource Sharing) is necessary because the frontend and backend run on different origins: the frontend on http://127.0.0.1:5500 and the backend on http://127.0.0.1:8000. The frontend includes the authorization header with the Bearer token in API requests, and the backend is configured to accept cross-origin requests from the frontend

origin. This configuration allows secure communication between the two services despite running on different ports.

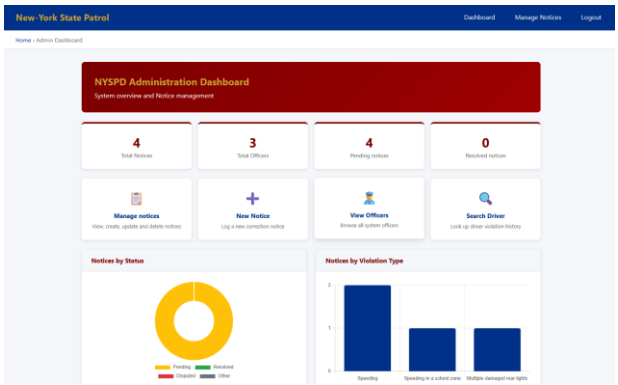
2.5 Deployment

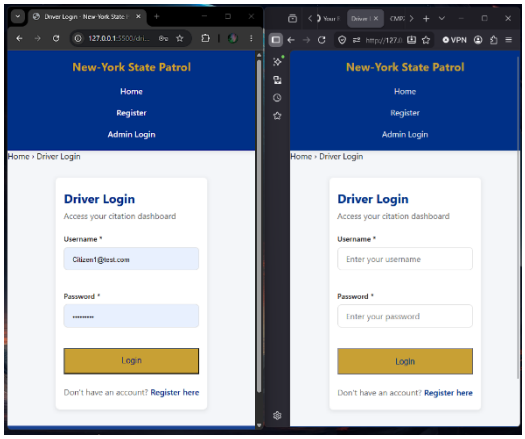
The frontend is deployed locally using VS Code's Live Server extension, which uses port 5500 as its default, this means you can find the frontend at: http://127.0.0.1:5500. Since the application uses vanilla HTML5, CSS3 and JavaScript without a build process or module bundling, deployment is straightforward: Open the index.html file with Live Server and the application should automatically open in your default browser.

3. Testing

3.1 Testing Approach

A thorough testing approach was taken to verify frontend functionality across multiple dimensions. Testing covered cross-browser compatibility, responsive design at different viewport sizes, API integration with both successful and failed requests, and complete user workflows.





3.1.1 Cross-Browser Testing

Cross-browser testing was conducted in google chrome and Mozilla Firefox to ensure compatibility across different rendering engines. Both browsers rendered the application identically, with form inputs, validation messages, modals, tables and API calls functioning consistently. JavaScript execution, CSS styling and event handling produced identical results in both browsers.

3.1.2 Responsive Design Testing

Responsive design was tested at 3 different viewport sizes. The home page displays correctly at mobile (320px) with single-column layout and stacked navigation, at tablet (768px) with two-column layouts, and at desktop (1920px) with full four-column layouts. CSS media queries successfully re-flow content at each breakpoint without changing the HTML structure.

3.1.3 API integration Testing

API integration was tested with both successful and failed scenarios. Successful API calls (login, notices, create notices) returned correct data and updated the UI appropriately. Error handling was verified by attempting failed requests, such as login with incorrect credentials, which correctly displayed error messages and prevented unauthorized access.

3.1.4 User Journey Testing

Complete user workflows were tested end-to-end. The driver journey (registration > login > dashboard > view notices) completed successfully with proper data display at each step. The admin journey (login > dashboard > create notice > view in table) similarly completed successfully, demonstrating that critical workflows function correctly across the application.

3.2 Reflection

3.2.1 Challenges

The biggest challenge during development was implementing JWT token authentication and session Management. Understanding how to decode tokens client-side, extracting role information and implementing automatic token expiry checking every 60 seconds required considerations around security and user experience. The solution involved storing tokens in sessionStorage, decoding them on login to determine user role, and running periodic expiry checks via setInterval() to automatically logout users when sessions expired.

3.2.2 What Worked Well

Several aspects of the implementation worked quite well, Modular JavaScript organization made debugging a lot easier, isolating functionality by type helped me find and solve any issues encountered. CSS customer properties at the :root level enabled consistent theming throughout the application. The desktop-first approach will CSS grid and Flexbox adapted layouts across mobile, tablet and desktop viewports. Real-time validation with visual feedback provided immediate user guidance during data entry.

3.2.3 Improvements

Given more time to work on this assignment, I would have expanded the registration forms to include more information gathering, it states in the brief that more inputs were expected however I couldn't figure out any workarounds for how I set up my tables in assignment 1 so that I could enter all that data without having to go into 17+ input fields.

3.2.4 Future Enhancements

This assessment provided me with deep learning in three key areas; JavaScript, JWT token authentication and API integration. In the future I would like to enhance this assignment by implementing a more detailed registration form, but also, I would like to have created a more detailed notice information table for the user.

---

4. References

Mozilla, (2026) *MDN, web docs.* Available at: https://developer.mozilla.org/en-US/ [Accessed 25th April 2026]

Mozilla, (2026) *MDN, Fetch API*. Available at: https://developer.mizilla.org/en-us/docs/web/API/Fetch_API [Accessed 25th April 2026]

Mozilla, (2026) *MDN, Web Storage API.* Available at: https://developer.mozilla.org/en-US/docs/web/API/Web_storage_API [Accessed 25th April 2026]